

Assessment: Transitioning from Azure ML Compute Instances to Compute Clusters

This report evaluates the proposed shift from individual **Compute Instances** to a shared **Compute Cluster** for our team of 20 data scientists. The primary goal is to resolve current job submission failures while maintaining our security policy against root access on user workstations.

1. Application Level Assessment

The daily workflow for data scientists will remain largely centered in the Azure ML Studio UI or VS Code. Instead of executing code directly on their private workstation, they will "submit" their work to the shared cluster. This change allows the UI to remain responsive even during heavy model training, as the heavy lifting is offloaded to the cluster nodes.

Feature	Compute Instance (Current)	Compute Cluster (Proposed)
Interactive Coding	Done directly on the instance.	Done in UI, offloaded to cluster.
Job Execution	Runs locally; blocks the UI.	Runs on cluster; UI stays free.
Environment Setup	Fails due to lack of root access.	Succeeds via Cluster Managed Identity.
User Experience	Individual workstation feel.	Shared "supercomputer" feel.

Practical Example: A scientist currently tries to run a training cell and receives an error because the instance cannot pull a Docker image from the Microsoft Container Registry (MCR). With the new setup, the scientist clicks "Run," the job is sent to the cluster, and the cluster automatically pulls the required image using its backend permissions, completing the task without user intervention.

2. Infrastructure Level Assessment

Code isolation is maintained through the workspace's folder structure, where notebooks are stored in individual `Users/<username>` directories. While the cluster is shared, each job runs in its own isolated Docker container, ensuring that one scientist's execution does not interfere with another's.

Scenario	Infrastructure Response	Cost Impact
----------	-------------------------	-------------

Single Job	One node spins up.	Pay for one VM only during the job.
20 Concurrent Jobs	Cluster autoscales to 20 nodes.	Pay for 20 VMs only during execution.
Idle Time	Cluster scales down to 0 nodes.	Zero cost during idle periods.
Code Conflict	Jobs take code snapshots.	No risk of overwriting active runs.

Worst-Case Scenario: If all 20 scientists run intensive schedules simultaneously, the cluster will provision 20 nodes to handle the load. The cost will be strictly linear—20 times the hourly rate of the selected VM—but only for the exact duration of the jobs. This is significantly more cost-effective than maintaining 20 active Compute Instances that accrue costs even when idle.

3. Solving the Job Submission Issue

The current job submission error is a direct result of our security policy restricting root access on Compute Instances. This restriction prevents the local Docker daemon from building or pulling images. By moving to a Compute Cluster, we can authorize the cluster's **Managed Identity** to access the MCR. This solves the "learning tasks" and training problems by moving the privileged operations to a secure, managed backend rather than the user's interactive workstation.

4. Implementation and POC Timeline

The transition requires minimal changes to existing notebooks. Data scientists only need to update their compute target name in their scripts. All existing notebooks and data stored in the workspace remain unaffected and do not need to be moved.

Phase	Activity	Estimated Time
Phase 1: Setup	Create cluster and configure Managed Identity.	2 Hours
Phase 2: POC	Test MCR image pulling with 2 scientists.	4 Hours
Phase 3: Rollout	Update compute targets in team notebooks.	1 Day
Total Duration	Full transition for the team.	~2 Days

Recommendation: We should initiate a 48-hour Proof of Concept (POC) focusing on the MCR image pull success. This will confirm that the cluster solves the submission error before we decommission the individual compute instances.